

---

# CS771 Assignment 1: Hamming PUF

---

## Group name: melody

Harshit Karnani (Roll No. 240440)

Anshit Goyal (Roll No. 240146)

Utkarsh Singhal (Roll No. 241119)

Vineesh Dubey (Roll No. 241167)

## Abstract

This report derives a linear feature map for the Hamming PUF, computes its dimensionality, and evaluates a LinearSVC model.

## 1 Part 1 - Detailed Mathematical Derivation

Let

$$m_i = c_i \oplus s_i,$$

where  $c_i$  denotes the  $i$ -th challenge bit and  $s_i$  denotes the  $i$ -th secret bit.

Define

$$h_e = \sum_{i \in E} m_i, \quad h_o = \sum_{j \in O} m_j,$$

where,

$$E = \{0, 2, 4, \dots, 30\}$$

and

$$O = \{1, 3, 5, \dots, 31\}.$$

The Hamming PUF output:

$$r(c) = \begin{cases} 1, & h_e h_o \geq t, \\ 0, & h_e h_o < t. \end{cases}$$

### 1.1 Linearization of the Hamming PUF

Using the identity

$$c_i \oplus s_i = c_i + s_i - 2c_i s_i,$$

we can rewrite  $m_i$  as

$$m_i = (1 - 2s_i)c_i + s_i.$$

Preprint.

Define

$$a_i = 1 - 2s_i.$$

Since  $s_i \in \{0, 1\}$ , it follows that

$$a_i \in \{-1, +1\}.$$

Therefore,

$$m_i = a_i c_i + s_i.$$

Substituting into the definition of  $h_e$ ,

$$h_e = \sum_{i \in E} m_i = \sum_{i \in E} (a_i c_i + s_i) = \sum_{i \in E} a_i c_i + \sum_{i \in E} s_i.$$

Let

$$S_E = \sum_{i \in E} s_i.$$

Then

$$h_e = S_E + \sum_{i \in E} a_i c_i.$$

Similarly,

$$h_o = S_O + \sum_{j \in O} a_j c_j,$$

where

$$S_O = \sum_{j \in O} s_j.$$

Expanding the product  $h_e h_o$  gives

$$h_e h_o = \left( S_E + \sum_{i \in E} a_i c_i \right) \left( S_O + \sum_{j \in O} a_j c_j \right).$$

Multiplying out the terms,

$$h_e h_o = S_E S_O + S_E \sum_{j \in O} a_j c_j + S_O \sum_{i \in E} a_i c_i + \sum_{i \in E} \sum_{j \in O} a_i a_j c_i c_j.$$

Observe that the resulting expression contains only a constant term, linear terms in  $c_i$ , and quadratic terms of the form  $c_i c_j$ .

## 1.2 Feature Map Construction

Motivated by the expansion derived above, we define the feature map

$$\phi : \{0, 1\}^{32} \rightarrow \mathbb{R}^{288}$$

by

$$\phi(c) = (c_0, \dots, c_{31}, \{c_i c_j\}_{i \in E, j \in O}).$$

The first 32 coordinates correspond to the original challenge bits, while the remaining coordinates correspond to all even-odd pairwise products  $c_i c_j$ .

Since there are 16 even indices and 16 odd indices, the number of quadratic features is

$$16 \times 16 = 256.$$

Hence,

$$D = 32 + 256 = 288.$$

We now construct a linear classifier in this feature space.

For the linear features  $c_i$ , define the corresponding weights as

$$w_i = \begin{cases} S_O a_i, & i \in E, \\ S_E a_i, & i \in O. \end{cases}$$

For each quadratic feature  $c_i c_j$ , where  $i \in E$  and  $j \in O$ , define

$$w_{ij} = a_i a_j.$$

Finally, define the bias term

$$b = S_E S_O - t.$$

Using these definitions, the expansion obtained previously can be written compactly as

$$w^\top \phi(c) + b = h_e h_o - t.$$

Therefore,

$$w^\top \phi(c) + b \geq 0 \iff h_e h_o \geq t.$$

By the definition of the Hamming PUF,

$$r(c) = 1 \iff h_e h_o \geq t.$$

Consequently,

$$r(c) = \frac{1 + \text{sign}(w^\top \phi(c) + b)}{2}.$$

Thus, the Hamming PUF can be represented exactly as a linear classifier in the transformed feature space  $\phi(c)$ .

### 1.3 Final Result

We have constructed a feature map

$$\phi : \{0, 1\}^{32} \rightarrow \mathbb{R}^{288}$$

given by

$$\phi(c) = \left( c_0, \dots, c_{31}, \{c_i c_j\}_{i \in E, j \in O} \right)$$

The corresponding weight vector is defined by

$$w_i = \begin{cases} S_O a_i, & i \in E, \\ S_E a_i, & i \in O, \end{cases}$$

for the linear features, and

$$w_{ij} = a_i a_j, \quad i \in E, j \in O,$$

for the quadratic features.

The bias term is

$$b = S_E S_O - t.$$

## 2 Part 2 - Dimensionality of Feature Space

The feature map constructed in Question 1 contains:

- 32 linear features  $c_0, \dots, c_{31}$ ,
- $16 \times 16 = 256$  quadratic features of the form  $c_i c_j$ , where  $i \in E$  and  $j \in O$ .

Therefore, the dimensionality of the feature space is

$$D = 32 + 256 = 288.$$

Hence,

$$D = 288.$$

## 3 Part 3

Submitted submit.py through google forms.

## 4 Part 4 - Experimental Results on Public Dataset

We evaluated the proposed feature map on the public Hamming PUF dataset. The original challenge vectors have dimension 32. After applying the map  $\phi$  described in Part 1, each challenge is transformed into a 288-dimensional feature vector. The public training set contains 7500 examples and the public test set contains 2500 examples.

We trained two linear models on the mapped training set: `LinearSVC` and `LogisticRegression`. For each experiment, we recorded the training time, training accuracy, and test accuracy. Training time was measured using Python's `time.perf_counter()` function. For consistency with our submitted hyperparameters, we used `max_iter=50` in these experiments.

#### 4.1 Effect of Loss Function in LinearSVC

| Loss          | Training Time (s) | Train Accuracy | Test Accuracy |
|---------------|-------------------|----------------|---------------|
| hinge         | 0.0536            | 0.9987         | 0.9944        |
| squared_hinge | 0.0741            | 0.9996         | 0.9968        |

Table 1: Effect of loss function on LinearSVC.

The squared\_hinge loss gave better training and test accuracy than the hinge loss in our experiment. Although it took slightly more training time, it achieved a test accuracy of 0.9968. Hence, squared\_hinge is a good practical choice for the submitted model.

#### 4.2 Effect of $C$ in LinearSVC

| $C$  | Training Time (s) | Train Accuracy | Test Accuracy |
|------|-------------------|----------------|---------------|
| 0.01 | 0.0511            | 0.9855         | 0.9740        |
| 0.1  | 0.0552            | 0.9957         | 0.9876        |
| 1    | 0.0551            | 0.9996         | 0.9968        |
| 10   | 0.0904            | 1.0000         | 0.9960        |
| 100  | 0.0904            | 1.0000         | 0.9960        |

Table 2: Effect of regularization parameter  $C$  on LinearSVC.

For LinearSVC, increasing  $C$  improved both training and test accuracy up to  $C = 1$ . Very large values of  $C$  achieved perfect training accuracy, but did not improve test accuracy further. This suggests that  $C = 1$  gives a good balance between fit quality and generalization.

#### 4.3 Effect of $C$ in LogisticRegression

| $C$  | Training Time (s) | Train Accuracy | Test Accuracy |
|------|-------------------|----------------|---------------|
| 0.01 | 0.0582            | 0.9665         | 0.9568        |
| 0.1  | 0.0800            | 0.9848         | 0.9724        |
| 1    | 0.1212            | 0.9953         | 0.9828        |
| 10   | 0.1837            | 0.9997         | 0.9920        |
| 100  | 0.2615            | 1.0000         | 0.9948        |

Table 3: Effect of regularization parameter  $C$  on LogisticRegression.

For LogisticRegression, increasing  $C$  consistently improved accuracy. This is because larger  $C$  corresponds to weaker regularization, allowing the model to fit the mapped features more closely. However, the training time also increased as  $C$  became larger.

#### 4.4 Effect of Tolerance in LinearSVC

| Tolerance | Training Time (s) | Train Accuracy | Test Accuracy |
|-----------|-------------------|----------------|---------------|
| $10^{-1}$ | 0.0650            | 0.9856         | 0.9752        |
| $10^{-2}$ | 0.0577            | 0.9987         | 0.9936        |
| $10^{-3}$ | 0.0557            | 0.9996         | 0.9968        |
| $10^{-4}$ | 0.0617            | 0.9996         | 0.9968        |
| $10^{-5}$ | 0.0821            | 0.9996         | 0.9968        |

Table 4: Effect of tolerance on LinearSVC.

Reducing the tolerance makes the convergence criterion stricter. We observe that lowering the tolerance improves accuracy up to  $10^{-3}$ , after which the test accuracy saturates. Smaller tolerance values do not provide additional accuracy gains.

#### 4.5 Effect of Tolerance in LogisticRegression

| Tolerance | Training Time (s) | Train Accuracy | Test Accuracy |
|-----------|-------------------|----------------|---------------|
| $10^{-1}$ | 0.0490            | 0.9841         | 0.9724        |
| $10^{-2}$ | 0.0838            | 0.9945         | 0.9828        |
| $10^{-3}$ | 0.1305            | 0.9953         | 0.9828        |
| $10^{-4}$ | 0.1392            | 0.9955         | 0.9836        |
| $10^{-5}$ | 0.1958            | 0.9955         | 0.9836        |

Table 5: Effect of tolerance on LogisticRegression.

For LogisticRegression, decreasing the tolerance generally increased training time. The test accuracy improved from  $10^{-1}$  to  $10^{-4}$ , but then saturated. Thus, very small tolerance values gave little additional benefit.

#### 4.6 Effect of Penalty

| Model              | Penalty  | Training Time (s) | Train Accuracy | Test Accuracy |
|--------------------|----------|-------------------|----------------|---------------|
| LogisticRegression | $\ell_1$ | 0.4192            | 0.9964         | 0.9836        |
| LogisticRegression | $\ell_2$ | 0.0981            | 0.9953         | 0.9828        |
| LinearSVC          | $\ell_1$ | 0.2101            | 0.9915         | 0.9800        |
| LinearSVC          | $\ell_2$ | 0.0558            | 0.9996         | 0.9968        |

Table 6: Effect of  $\ell_1$  and  $\ell_2$  penalties.

The  $\ell_1$  penalty generally required more training time than the  $\ell_2$  penalty. For LogisticRegression, both penalties gave similar test accuracy, but  $\ell_2$  was much faster. For LinearSVC,  $\ell_2$  gave much better test accuracy and was also faster. Therefore,  $\ell_2$  regularization is preferable for this task.

#### 4.7 Conclusion

The experiments show that the proposed feature map is effective for breaking the Hamming PUF. After mapping the original 32-dimensional challenges to 288-dimensional features, both LinearSVC and LogisticRegression achieve high test accuracy. Among the tested models, LinearSVC with  $C = 1$ , tolerance  $10^{-3}$ , squared\_hinge loss, and  $\ell_2$  penalty provides a strong tradeoff between accuracy and training time, achieving training accuracy 0.9996 and test accuracy 0.9968 on the public dataset.

### 5 Part 5 - Effect of Training Set Size

We studied how many challenge-response pairs are required to learn the Hamming PUF accurately. For this experiment, we kept the public test set fixed and randomly selected smaller training subsets from the original public training set. The subset sizes used were 100, 200, 300, 500, 1000, 1500, 2000, 3000, 4000, and 5000. For each subset size, we trained a LinearSVC model using the 288-dimensional feature map described earlier.

The hyperparameters used were  $C = 1$ , tolerance  $10^{-3}$ , squared\_hinge loss,  $\ell_2$  regularization, and max\_iter=50. The public test set was kept unchanged for all trials.

| Train Size | Training Time (s) | Train Accuracy | Test Accuracy |
|------------|-------------------|----------------|---------------|
| 100        | 0.0014            | 1.0000         | 0.8084        |
| 200        | 0.0016            | 1.0000         | 0.8752        |
| 300        | 0.0081            | 1.0000         | 0.8884        |
| 500        | 0.0078            | 1.0000         | 0.9068        |
| 1000       | 0.0097            | 1.0000         | 0.9240        |
| 1500       | 0.0148            | 1.0000         | 0.9404        |
| 2000       | 0.0143            | 1.0000         | 0.9460        |
| 3000       | 0.0220            | 1.0000         | 0.9732        |
| 4000       | 0.0355            | 1.0000         | 0.9860        |
| 5000       | 0.0428            | 1.0000         | 0.9904        |

Table 7: Effect of training set size on train and test accuracy.

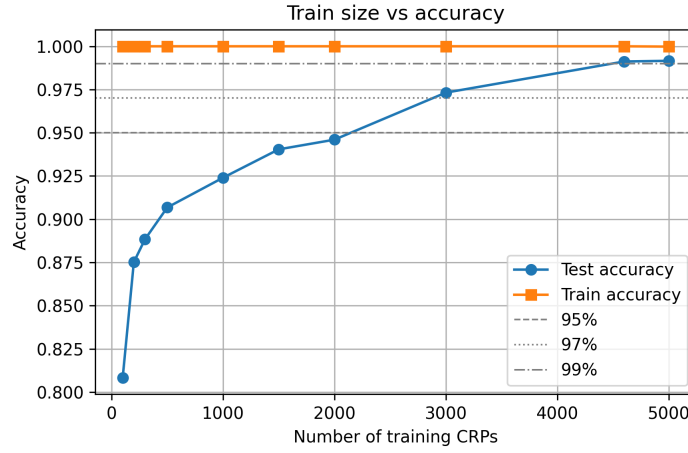


Figure 1: Train size vs train and test accuracy for LinearSVC.

The results show that very small training sets are not sufficient for accurately learning the PUF. With only 100 CRPs, the test accuracy is 0.8084. As the number of training CRPs increases, the test accuracy improves steadily. The training accuracy remains close to 1.0 for all tested subset sizes, but this does not imply good generalization for small datasets. For example, with 100 training CRPs the model fits the selected training subset perfectly, but the test accuracy is only 0.8084.

To estimate the minimum number of CRPs required for specific accuracy levels, we also performed a fine-grained threshold search. Starting from 100 CRPs, we increased the training size 100 points at a time using a fixed random ordering of the training examples. The same LinearSVC hyperparameters were used.

| Target Test Accuracy | Minimum Training CRPs | Observed Test Accuracy |
|----------------------|-----------------------|------------------------|
| 95%                  | 2000                  | 0.9492                 |
| 97%                  | 3100                  | 0.9708                 |
| 99%                  | 4600                  | 0.9912                 |

Table 8: Minimum number of training CRPs required to reach different test accuracy levels.

Thus, the full set of 7500 training CRPs is not strictly necessary to obtain high accuracy on the public test set. In our experiment, 2000 CRPs were enough to reach 95% test accuracy, 3100 CRPs were enough to reach 97% test accuracy, and 4600 CRPs were enough to reach 99% test accuracy.

## References

1. P.Kar, "CS771 Lecture 1: Welcome + Features and Models", IIT Kanpur, 2025-26. Available at: <https://www.cse.iitk.ac.in/users/purushot/courses/ml/2025-26-s/material/lectures/lec1.pptx>
2. LinearSVC Documentation. Available at: <https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html>
3. LogisticRegression Documentation. Available at: [https://scikit-learn.org/stable/modules/generated/sklearn.linear\\_model.LogisticRegression.html](https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html)